Frontend Interview Preparation Handbook

HTML, CSS, JavaScript, TypeScript, React, Next.js, and the web platform

Purpose. This is a practical study guide for frontend interviews. It is designed to help you explain fundamentals, write working code, reason about tradeoffs, debug production problems, and communicate clearly under pressure.

Important. No document can guarantee an offer. Interview outcomes also depend on the role, the interview loop, communication, and practice. This handbook is intentionally broad so that you can prepare for common junior, mid-level, and senior frontend topics without memorizing isolated answers.

How to use it: Read the foundations once, then solve the practical questions without looking at the answers. For each topic, practise explaining the why, not just the definition. Use the final revision sheet the day before the interview.

Contents and study route

1. Web platform foundations: browser rendering, HTTP, DOM, events, storage, and accessibility.
2. HTML and CSS: semantic markup, layout, responsive design, animation, and visual debugging.
3. JavaScript: language behavior, the event loop, asynchronous work, modules, and performance.
4. TypeScript: narrowing, generics, utility types, API contracts, and runtime boundaries.
5. React: rendering, state, effects, hooks, forms, performance, testing, and architecture.
6. Next.js: App Router, server and client components, data fetching, caching, routing, and deployment.
7. Production frontend: accessibility, performance, security, testing, observability, and system design.
8. Practice bank: MCQs, coding exercises, debugging cases, and real-world interview scenarios.
9. Final revision: short answers, checklists, and questions to ask the interviewer.

Part one Web platform foundations

A strong frontend interview answer starts with the browser, not the framework. React and Next.js are tools that eventually produce HTML, CSS, JavaScript, network requests, and browser work. If you can reason from those layers upward, framework questions become easier.

The browser request and render path

Core model: A browser resolves a URL, opens a connection, sends an HTTP request, receives a response, parses HTML, discovers and fetches resources, builds the DOM and CSSOM, creates a render tree, performs layout, paints pixels, and composites layers. JavaScript can interrupt or change parts of this pipeline.

- DNS maps a hostname to an address. The browser may reuse a connection, use HTTP/2 or HTTP/3, and negotiate TLS for HTTPS.
- HTML parsing creates the DOM. A normal synchronous script can pause parsing while it downloads and executes.
- CSS parsing creates the CSSOM. The browser generally needs enough CSS to determine styles before it can paint correctly.
- Layout calculates geometry. Paint creates text, borders, backgrounds, and images. Compositing combines layers, often using the GPU.
- A layout change can trigger layout and paint. A transform or opacity change is usually cheaper because it can often be composited without recalculating geometry.

Interview answer: If asked why a page is slow, separate network cost, JavaScript cost, style and layout cost, paint cost, and user-perceived latency. Do not answer only with a framework optimization.

HTTP and browser networking

Concept	What to know	Typical interview trap
GET	Reads a resource. It should be safe and cacheable when the response permits it.	Using GET for a mutation creates retries and caching problems.
POST	Creates or triggers an action. It is not automatically idempotent.	Assuming the client can safely retry every POST.
PUT	Replaces a resource at a known URI and is intended to be idempotent.	Treating PUT as a partial update without defining semantics.
PATCH	Applies a partial change. Idempotency depends on the patch operation.	Sending an increment operation and assuming retries are safe.

Concept	What to know	Typical interview trap
Cache-Control	Controls browser and intermediary caching behavior.	Caching private or user-specific data publicly.
CORS	A browser security rule for cross-origin requests. The server must opt in with response headers.	Thinking CORS protects the API from non-browser clients.
AbortController	Cancels a fetch from the client when the request is obsolete or the component is leaving.	Calling abort and treating it as a product error.

1. What is the difference between CORS and authentication?

answer: CORS controls whether browser JavaScript may read a cross-origin response. Authentication identifies the caller. An authenticated request can still fail CORS, and a CORS-allowed request can still be unauthenticated.

why: CORS is enforced by browsers and is not an access-control mechanism for the server. The API must still validate tokens, sessions, permissions, and input.

interviewer follow-up: How would you allow a frontend at app.example.com to call api.example.com without allowing every origin?

follow-up answer: Keep an exact server-side allowlist of trusted origins. When the request Origin matches, return that origin in Access-Control-Allow-Origin, add Vary: Origin, and allow credentials only when the cookie and CSRF design support it. Handle the OPTIONS preflight with the narrow method and header list.

why: CORS is a browser read permission, not authentication. Exact origin matching prevents an unrelated website from reading responses while the server still performs authentication and authorization.

2. How would you prevent stale search results from overwriting newer results?

answer: Track a request id or abort the previous request when the query changes. Only commit a response if it still belongs to the current query.

why: Network responses can finish in a different order from the order in which they were started. Debouncing reduces traffic but does not solve response races by itself.

interviewer follow-up: What should the UI show when the request is aborted versus when the server returns an error?

follow-up answer: Treat an intentional abort as a silent transition to the next request or idle state. For a server error, preserve the input, show a short recoverable message, expose retry, and keep enough context for support diagnostics.

why: An abort usually means the user made a newer request or left the screen; showing an error for it creates noise. A server error is actionable and should not look like success.

```
const controllerRef = useRef<AbortController | null>(null);

async function search(query: string) {
  controllerRef.current?.abort();
  const controller = new AbortController();
  controllerRef.current = controller;

  try {
    const response = await fetch(`/api/search?q=${encodeURIComponent(query)}`, {
      signal: controller.signal,
    });
    if (!response.ok) throw new Error(`http ${response.status}`);
    return await response.json();
  } catch (error) {
    if (error instanceof DOMException && error.name === 'AbortError') return null;
    throw error;
  }
}
```

The DOM, events, and the event loop

DOM: The DOM is an object representation of the document. It is not the same thing as the pixels on screen. Updating it may cause style recalculation, layout, paint, or compositing work.

Event propagation: An event travels through capture, reaches the target, then travels through bubble. Event delegation attaches one listener to a stable ancestor and uses the event target to handle many children.

Event loop: JavaScript runs on a call stack. After the current task finishes, the runtime drains microtasks such as promise callbacks before it moves to another task such as a timer or input event. Rendering opportunities occur between tasks when the browser can paint.

```
console.log('a');
setTimeout(() => console.log('timer'), 0);
queueMicrotask(() => console.log('microtask'));
Promise.resolve().then(() => console.log('promise'));
console.log('b');

// output: a, b, microtask, promise, timer
```

3. Why can a promise callback run before a zero millisecond timer?

answer: Promise reactions are microtasks. Microtasks are drained after the current JavaScript task and before the event loop proceeds to the timer task.

why: The number zero on a timer means the timer becomes eligible after its minimum delay; it does not interrupt the current task or jump ahead of microtasks.

interviewer follow-up: What happens if a microtask keeps scheduling another microtask forever?

follow-up answer: The browser can remain in microtask processing and fail to reach timers, input, or a paint opportunity. Break the chain by yielding with setTimeout, requestAnimationFrame, a worker, or a bounded loop.

why: Microtasks run before the event loop proceeds to the next task. An unbounded chain can starve rendering and make the page appear frozen even though JavaScript is still running.

Storage, cookies, and browser security

Storage	Good use	Important limitation
Cookies	Server sessions, small request metadata, and security attributes such as HttpOnly and SameSite.	Small size and sent with matching requests; exposed to CSRF risk if configured poorly.
localStorage	Small non-sensitive preferences that can survive browser restarts.	Synchronous API and readable by JavaScript, so an XSS bug can expose it.
sessionStorage	Short-lived per-tab state.	Not shared across tabs and still readable by JavaScript.
IndexedDB	Larger structured offline data and caches.	More complex lifecycle and still subject to origin security.
Cache Storage	Service worker and response caching.	Must define invalidation and avoid serving stale private data.

- For a sensitive session cookie, consider Secure, HttpOnly, SameSite, narrow Path and Domain, short lifetime, rotation, and server-side invalidation.
- Do not describe localStorage as secure storage. It is convenient storage, not a protection boundary.
- Never put a long-lived secret such as a database service key in frontend code. Anything shipped to the browser is observable by the user.
- Use a Content Security Policy and output encoding to reduce XSS risk. Validate authorization on the server even when the UI hides a control.

Accessibility is a product requirement

Semantic first: Use a real button for an action, a link for navigation, a label for a form control, and headings that represent the document outline. A div with a click handler is not an equivalent button.

- Keyboard users need focus order, visible focus, Escape behavior for dismissible UI, and no keyboard traps.
- Screen readers need accessible names, useful roles, state announcements, and meaningful alternative text.

- Do not communicate state with color alone. Check contrast, reduced motion, zoom, text resizing, and touch target size.
- Test the actual interaction: tab through the page, submit forms without a mouse, use a screen reader when possible, and check reduced-motion behavior.

4. How do you make a modal accessible?

answer: Move focus into the dialog, give it an accessible name, keep focus inside while it is open, close on Escape when appropriate, restore focus to the trigger, and ensure the background is not announced or interacted with.

why: A modal changes the user's focus context. Visual styling alone does not create the interaction contract that keyboard and assistive technology users need.

interviewer follow-up: How would you handle a modal that contains a form and reports validation errors?

follow-up answer: Keep focus in the dialog, focus the first invalid field or an error summary, connect each message with aria-describedby, announce the summary when appropriate, and preserve valid input. Closing still restores focus to the trigger.

why: Validation changes the focus and announcement model. A user should know what failed without losing their work or being pushed behind the dialog.

Part two HTML

HTML is the structure and meaning of the interface. Interviewers often use HTML questions to see whether you build a usable document before layering on JavaScript and styling.

Semantic HTML and document structure

Element	Use it for	Common mistake
header	Introductory content for a page or section.	Assuming there can be only one header on a page.
nav	A group of major navigation links.	Wrapping every collection of links in nav.
main	The unique primary content of the document.	Using multiple main elements without a valid hidden pattern.
section	A themed grouping, usually with a heading.	Using section as a generic styling wrapper.
article	A self-contained piece that could	Using article only because the

Element	Use it for	Common mistake
	stand alone or be syndicated.	design has a card.
button	An action such as opening, submitting, toggling, or deleting.	Using an anchor or div for an action with no keyboard behavior.
a	Navigation to a URL or document location.	Using href='javascript:' or an anchor without a meaningful destination.

```
<main>
  <h1>Account settings</h1>
  <section aria-labelledby="profile-heading">
    <h2 id="profile-heading">Profile</h2>
    <form>
      <label for="display-name">Display name</label>
      <input id="display-name" name="displayName" autocomplete="name" />
      <button type="submit">Save changes</button>
    </form>
  </section>
</main>
```

5. When should you use a button instead of a link?

answer: Use a button when the action changes state or performs an operation. Use a link when the user is going somewhere and the destination can be represented by a URL.

why: Native semantics provide keyboard behavior, accessibility information, focus handling, and predictable browser behavior. Recreating those behaviors is easy to get wrong.

interviewer follow-up: What if the link opens a menu instead of navigating?

follow-up answer: Use a button with an accessible name, aria-expanded, and aria-controls. Use a link only when activation has a meaningful destination. If an item both navigates and opens a menu, separate the controls so each has one clear purpose.

why: Native semantics communicate the expected keyboard behavior. A link that does not navigate creates a misleading interaction for keyboard and assistive technology users.

6. What is the difference between alt text and a figure caption?

answer: Alt text is a concise text alternative for users who cannot perceive the image. A figure caption is visible context for all users and may explain why the image matters.

why: They serve different audiences and purposes. Decorative images should usually have an empty alt attribute, while informative images need a meaningful alternative.

interviewer follow-up: How would you write alt text for a chart that also has a data table?

follow-up answer: Give the chart concise alt text that states its purpose and the main takeaway, then provide the complete data in the visible table. Example: 'Monthly signups increased from January through April; detailed values are in the table below.'

why: Alt text should not force a screen-reader user through a long data dump when an equivalent structured table is available. The visible table also helps everyone inspect exact values.

Forms and input behavior

- Use labels, autocomplete hints, input types, constraints, and server-side validation. Client validation improves feedback but is not a security boundary.
- Preserve user input on recoverable errors. Announce validation errors and connect them to fields with aria-describedby or a clear error summary.
- Choose controlled or uncontrolled form inputs deliberately in React. Controlled inputs simplify derived UI but can cause unnecessary work in very large forms.
- Handle composition events for international text input. Do not assume every input event represents a completed character or word.
- Be careful with file inputs, drag and drop, paste, large files, slow uploads, cancellation, retries, and duplicate submissions.

Part three CSS

CSS interviews are usually about reasoning: which rule wins, how layout is calculated, why an element overflows, and how to build a responsive interface without fragile offsets.

Cascade, specificity, and the box model

Cascade: The winning declaration depends on origin and importance, cascade layers, specificity, scoping, and source order. Specificity is not simply a universal score; the selector comparison is performed by components.

Box model: By default, width and height apply to the content box. With box-sizing: border-box, the declared size includes padding and border. This is usually easier for predictable layouts.

```
*, *::before, *::after {
  box-sizing: border-box;
}

:root {
  --content-width: min(100% - 2rem, 72rem);
}

.page {
  width: var(--content-width);
  margin-inline: auto;
}
```

7. Why is !important usually a maintenance problem?

answer: It overrides normal cascade decisions and makes future changes harder to reason about. Prefer a better component boundary, a cascade layer, a more precise state selector, or corrected source order.

why: The problem is not that !important never works; it is that it hides the reason a rule wins and creates a specificity escalation cycle.

interviewer follow-up: Name a legitimate use of !important.

follow-up answer: A constrained example is a user-controlled accessibility override or a utility that must override an embedded third-party style when the integration cannot be changed. Keep it isolated, documented, and covered by a test.

why: The issue is uncontrolled specificity escalation. A narrow, intentional override can be valid, but normal application styles should use clear layers and ownership instead.

Flexbox, grid, and responsive layout

Tool	Best fit	Useful mental model
Flexbox	One-dimensional rows or columns, alignment, and distribution.	A line of items that negotiate available space.
Grid	Two-dimensional page or component layout.	Rows and columns with explicit or automatic tracks.
Block flow	Text and sections that should naturally stack.	Let content determine height; avoid unnecessary positioning.
Absolute positioning	Overlays, badges, anchored decorative elements, or a controlled layer.	Remove an element from normal flow; define its containing block deliberately.
Container queries	Components that respond to their own available space.	The component adapts to its container, not only the viewport.

- Use minmax(), min(), max(), clamp(), auto-fit, and intrinsic sizing to let content participate in layout.
- Test narrow phones, wide monitors, zoomed text, long translations, empty states, loading states, and user-generated content.
- Avoid fixed heights for text-heavy content. A fixed format such as an icon button can use stable dimensions, but content containers should usually grow.
- Use gap for relationships between flex and grid children rather than scattered margins that create edge-case collapse behavior.

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 16rem), 1fr));
  gap: 1rem;
}

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```

8. Why does z-index sometimes appear not to work?

answer: z-index is ordered within stacking contexts. A child cannot escape its parent's stacking context just because it has a larger z-index than an element in a different context.

why: Transforms, opacity less than one, positioned elements with z-index, isolation, filters, and some containment properties can create stacking contexts. Debug the context tree, not only the numeric value.

interviewer follow-up: How would you debug a dropdown hidden behind a transformed parent?

follow-up answer: Inspect the ancestor chain for transform, opacity, filter, isolation, containment, and positioned z-index values. Identify the stacking context and clipping ancestor, then render the dropdown in an appropriate portal or remove the unnecessary context.

why: z-index values only compete within compatible stacking contexts. A portal can move an overlay to a context where it can be ordered and clipped correctly.

9. How do you center an element without hard-coding its position?

answer: Choose the layout mechanism that matches the relationship, such as display flex with align-items and justify-content, or grid with place-items. Use margins and intrinsic sizing where appropriate.

why: Hard-coded top and left values break when content, viewport size, font metrics, or writing direction changes.

interviewer follow-up: What changes when the centered item is taller than the viewport?

follow-up answer: Do not force mathematical vertical centering that hides the top or bottom. Use a scrollable page with safe padding, align toward the top when content exceeds the viewport, and test zoom and large text.

why: A layout must preserve access to all content. Centering is a visual preference; it must yield to readable, scrollable content.

Animation and visual performance

- Prefer transform and opacity for frequent motion. Changing width, height, top, left, font size, or box-shadow can be more expensive, though the right choice depends on the scene.
- Do not add will-change everywhere. It can increase memory use and create too many compositor layers.
- Use prefers-reduced-motion, avoid flashing content, and keep focus and reading order stable while elements animate.
- When a layout jitters, look for changing intrinsic sizes, late-loading fonts, image dimensions that were not reserved, scrollbars appearing, and state updates that remount content.

Part four JavaScript

JavaScript questions often test whether you understand runtime behavior rather than whether you remember syntax. Explain what runs, when it runs, what is copied, and what can be observed at runtime.

Types, coercion, scope, and closures

Topic	Strong answer	Edge case
let and const	Block-scoped bindings. const prevents rebinding, not object mutation.	A const object can still have its properties changed.
var	Function-scoped and hoisted with an initialized value of undefined.	Redeclaration and loop closure behavior can surprise you.
==	Coercive equality with conversion rules.	Avoid for most application logic unless you can explain the conversion.
===	Strict equality without ordinary type coercion.	NaN is not equal to itself; use Number.isNaN.
Object.is	Same-value comparison with different NaN and signed-zero behavior.	Object.is(-0, 0) is false.
Closure	A function retains access to variables from its lexical environment.	Long-lived callbacks can retain more memory than intended.

```
function createCounter() {
  let count = 0;
  return () => ++count;
}

const next = createCounter();
next(); // 1
next(); // 2
```

10. What is a closure and where is it useful in frontend code?

answer: A closure is a function together with access to variables from the lexical scope where it was created. It is useful for factories, event handlers, memoized functions, private state, and callbacks.

why: The callback does not copy the variable's current value into a separate global; it keeps a reference to the lexical environment. This explains both its power and its memory implications.

interviewer follow-up: How can a closure create a stale value in a React effect?

follow-up answer: An effect callback retains the values from the render that created it. Include changing values in the dependency list, use a functional state update for derived next state, or use a ref only when the latest mutable value is intentionally needed.

why: The fix depends on ownership. Adding every value to dependencies keeps synchronization honest; a ref is not a replacement for state when the UI must update.

11. What is the difference between null and undefined?

answer: undefined commonly means a value was not provided or a property is absent. null is an explicit empty value. Application APIs should choose a consistent contract instead of relying on accidental distinctions.

why: Both are falsy, but they communicate different intent and interact differently with property access, JSON, default parameters, and validation.

interviewer follow-up: Why does JSON.stringify omit an object property whose value is undefined?

follow-up answer: JSON has null but no undefined object value, so the serializer omits that property. In arrays, an undefined entry becomes null. Convert the value explicitly when the API contract needs a distinction.

why: Serialization follows the JSON data model, not the full JavaScript value model. An API should define whether missing and null mean different things.

Objects, prototypes, and data structures

- Objects inherit through a prototype chain. Classes are syntax over prototype-based behavior, not a separate object model.
- Use Map when keys are not limited to strings or when insertion order and explicit key operations matter. Use Set for membership and deduplication.

- Choose arrays for ordered collections, objects or Maps for lookup, and typed arrays for binary numeric data.
- Avoid mutating shared state when consumers depend on reference equality for change detection. Copy only the levels that change, or use a well-understood immutable update tool.

12. Why can mutating an object fail to update a UI?

answer: A UI library may compare the object reference and conclude that nothing changed. If the same object is mutated and passed back, shallow equality cannot detect the internal change.

why: Immutable updates create a new reference for changed data, making change detection predictable. This is a convention, not a magical requirement of JavaScript itself.

interviewer follow-up: What is the tradeoff of deep cloning every update?

follow-up answer: Deep cloning simplifies immutability but costs CPU and memory, can break object identity, and may mishandle Date, Map, Set, File, functions, or cyclic data. Prefer a targeted immutable update or a library with structural sharing.

why: The goal is predictable change detection, not copying every byte. Copy the changed path and preserve stable references where the data contract allows it.

Asynchronous JavaScript and fetch

- Promises represent a future result and can be pending, fulfilled, or rejected. Await pauses the async function, not the entire JavaScript thread.
- fetch rejects on network failures and aborts, but a 404 or 500 still resolves. Check response.ok or the status explicitly.
- Use Promise.all when all independent operations are required, Promise.allSettled when you need every outcome, and a sequential loop when order or rate limits matter.
- Use timeouts, cancellation, retry budgets, exponential backoff with jitter, and idempotency keys for operations that may be retried.
- Do not update state after a component is logically obsolete. Abort, sequence-check, or move the work into a data layer with explicit ownership.

```
async function loadUsers(signal: AbortSignal) {
  const response = await fetch('/api/users', {signal});
  if (!response.ok) {
    throw new Error(`request failed: ${response.status}`);
  }
  const data: unknown = await response.json();
  return parseUsers(data);
}
```

13. When should you use Promise.all versus Promise.allSettled?

answer: Use Promise.all when any failure should fail the combined operation and all results are required. Use Promise.allSettled when independent operations should each report success or failure.

why: The choice is a product and control-flow decision. A dashboard may show partial widgets, while a checkout operation may need an all-or-nothing result.

interviewer follow-up: How would you limit concurrency when loading 1,000 images?

follow-up answer: Use a queue with a small worker limit, such as four to eight active requests, and cancel or skip work that is no longer visible. Apply lazy loading, decode only near the viewport, and record failures separately.

why: Unbounded concurrency competes for bandwidth, memory, sockets, and decoding time. A bounded queue improves fairness and keeps the interface responsive.

Modules, errors, and performance

- ES modules have static imports and exports that bundlers can analyze. Prefer small explicit module boundaries over a global namespace.
- Catch errors where you can add context or recover. Do not catch and silently discard errors. Preserve the original cause when rethrowing.
- Measure before optimizing. Use the Performance panel, performance marks, Web Vitals, bundle analyzers, and realistic throttled devices.
- Avoid long synchronous work on the main thread. Split work, defer non-critical work, use requestIdleCallback carefully, or move CPU-heavy work to a worker.

```
try {
  await saveDraft();
} catch (error) {
  throw new Error('draft save failed', {cause: error});
}
```

Part five TypeScript

TypeScript improves the development-time contract. It does not validate untrusted JSON at runtime and it does not make a frontend secret. A strong answer keeps the type system and runtime boundary separate.

Inference, unions, and narrowing

Inference: TypeScript often derives types from values and function returns. Use explicit annotations where they clarify a public boundary, prevent widening, or document an important invariant.

Union: A union means a value may be one of several types. You narrow it with checks such as typeof, in, equality, discriminants, or user-defined type guards.

```
type Result<T> =
  | {status: 'ok'; data: T}
  | {status: 'error'; message: string};

function render(result: Result<User[]>) {
  if (result.status === 'error') return result.message;
  return result.data.map(user => user.name).join(', ');
}
```

14. What is the difference between any, unknown, and never?

answer: any disables most checking. unknown accepts any value but requires narrowing before use. never represents an impossible value or a function that cannot complete normally.

why: unknown makes the runtime boundary explicit, while any hides uncertainty. never is useful for exhaustiveness checks and APIs that do not return.

interviewer follow-up: How would you use never to detect a new union member during compilation?

follow-up answer: Use a discriminated union and pass the default branch to an assertNever function. When a new member is added but not handled, TypeScript reports that the value is no longer never.

why: The compiler becomes a checklist for exhaustive behavior. This prevents a new state from silently falling through to a misleading default UI.

```
function assertNever(value: never): never {
  throw new Error(`unexpected value: ${String(value)}`);
}

function label(status: 'idle' | 'loading' | 'done') {
  switch (status) {
    case 'idle': return 'Ready';
    case 'loading': return 'Loading';
    case 'done': return 'Complete';
    default: return assertNever(status);
  }
}
```

Generics and reusable contracts

Generic: A generic keeps a relationship between input and output types. It is more useful than using any when the caller should retain information about the type.

```
function first<T>(items: T[]): T | undefined {
  return items[0];
}

function getProperty<T, K extends keyof T>(item: T, key: K): T[K] {
  return item[key];
}
```

TypeScript topic	Interview-ready explanation
interface versus type	Both can describe object shapes. Interfaces are extendable and participate in declaration merging; type aliases compose unions, intersections, tuples, and mapped types naturally. Choose based on the API and team convention.
readonly	Prevents assignment through that type view. It does not deep-freeze a runtime object.
utility types	Mapped helpers such as Partial, Pick, Omit, Record, and ReturnType transform existing contracts; use them to keep related types aligned.
strict mode	Enables stronger checks such as strictNullChecks and noImplicitAny. It exposes bugs earlier and makes boundaries more explicit.
runtime validation	Use a parser or schema at external boundaries. A type assertion changes the compiler's view but does not inspect the value.

Part six React

React interviews are about predictable rendering and state ownership. The best answers explain what data changes, which component owns it, what must be synchronized with an external system, and how to measure performance.

Components, props, state, and rendering

- Props are inputs owned by a parent. State is data a component owns across renders. Derived values should usually be calculated during render instead of duplicated in state.
- A render is a calculation of the UI for a particular state and props. Rendering is not the same as committing DOM changes.
- Keys identify list items across renders. Use stable identity from the data, not the array index when items can be inserted, removed, or reordered.
- Lift state to the nearest common owner when multiple components need the same source of truth. Avoid lifting everything to the application root.
- Keep side effects out of render. Effects synchronize with external systems such as subscriptions, browser APIs, and imperative widgets.

15. Why is using an array index as a key risky?

answer: When the list changes, an index can point to a different item even though React sees the same key. State, focus, or input values can then appear to move to the wrong row.

why: A key should represent the identity of the item, not its current position. Index keys are acceptable only for truly static lists where order never changes.

interviewer follow-up: What if the backend does not provide an id?

follow-up answer: Ask whether the backend can provide a stable immutable id. If not, use a documented composite key only when its uniqueness and stability are guaranteed. Do not use an array index for data that can reorder.

why: React keys need identity across renders. A position is not identity, and a guessed key can cause state to move between records.

16. When should you use useEffect?

answer: Use it to synchronize React with something outside React: a subscription, timer, network connection, DOM API, or imperative library. Do not use it simply to derive a value from props or state.

why: An unnecessary effect adds another render cycle, creates dependency questions, and can cause stale data or loops. Derived values belong in render or a memo only when measurement shows a need.

interviewer follow-up: What cleanup is required for a subscription or timer?

follow-up answer: Return a cleanup function that unsubscribes, clears the timer or interval, aborts owned requests, disconnects observers, and removes event listeners with the same function and options used to add them.

why: Cleanup makes setup safe to repeat and prevents memory leaks, duplicate notifications, and updates after a feature is no longer active.

```
function SearchResults({query}: {query: string}) {
  const [state, setState] = useState<State>({status: 'idle'});

  useEffect(() => {
    const controller = new AbortController();
    setState({status: 'loading'});
    fetchResults(query, controller.signal)
      .then(data => setState({status: 'success', data}))
      .catch(error => {
        if (error.name !== 'AbortError') setState({status: 'error', error});
      });
    return () => controller.abort();
  }, [query]);

  return <ResultsView state={state} />;
}
```

Hooks and state design

Hook	Use it for	Do not use it for
useState	Local state that changes over time.	Mirroring every prop or storing cheap derived values.
useReducer	State transitions with several related fields or explicit events.	Making a two-value toggle harder to read.
useEffect	External synchronization and cleanup.	Pure calculations.
useMemo	Caching an expensive calculation when profiling supports it.	Making every expression memoized by default.
useCallback	Stable callback identity for a memoized child or hook contract.	Assuming it makes the function itself faster.
useRef	A mutable value or DOM reference that should not trigger a render.	Using it as a hidden state store that the UI must reflect.
useContext	Reading a value shared through a provider.	Replacing every state-management boundary with one giant context.

17. How do you avoid a context provider causing too many re-renders?

answer: Split contexts by update frequency and responsibility, memoize provider values when appropriate, keep provider state close to consumers, and use selector-based stores when the state graph requires it.

why: Every consumer of a context can be notified when its value identity changes. The solution is usually better ownership and granularity, not blindly wrapping everything in memo.

interviewer follow-up: How would you prove the provider is actually the bottleneck?

follow-up answer: Record the interaction with React DevTools Profiler, inspect which consumers render and why, compare provider value identity, and measure commit duration before changing the architecture.

why: Memoization without evidence adds complexity. Profiling distinguishes a provider problem from an expensive child, a broad parent update, or a slow browser task.

18. What is the difference between controlled and uncontrolled inputs?

answer: A controlled input gets its current value from React state and reports changes through an event. An uncontrolled input keeps its value in the DOM and can be read through a ref or form submission.

why: Controlled fields make validation and conditional UI straightforward. Uncontrolled fields can reduce render work and integrate naturally with native forms, especially in large forms.

interviewer follow-up: How would you handle a file input in a controlled form?

follow-up answer: Treat the file input as an uncontrolled browser-owned control, read its FileList from the change event or a ref, store metadata separately, validate size and type, and reset it by clearing the value or remounting when needed.

why: Browsers do not allow arbitrary code to set a local file path. Separating the native control from the form model respects that security boundary.

React performance and concurrency

- Start with profiling. Find the component that renders, why it renders, and whether the work is expensive or merely frequent.
- Use stable keys, avoid recreating large structures unnecessarily, colocate state, virtualize long lists, and split code at useful route or feature boundaries.
- memo, useMemo, and useCallback are tools for a measured identity problem. They add comparison and mental overhead.
- Transitions can mark non-urgent updates so urgent input remains responsive. They do not make CPU work disappear and do not replace cancellation for requests.
- Suspense coordinates loading boundaries. A fallback should be useful, stable, and not cause the whole page to jump unexpectedly.

19. What causes a React component to re-render?

answer: Its state changes, its parent renders and passes new props, a consumed context value changes, or an external store subscription notifies it. A re-render does not necessarily mean the DOM changes.

why: React can render to compute the next result, then compare and commit only needed changes. Separate render frequency from browser painting cost.

interviewer follow-up: How would you diagnose a component that renders too often?

follow-up answer: Use the React Profiler and a render-reason tool, check parent renders, prop and context identity, effect loops, unstable keys, and expensive calculations. Fix ownership or identity first, then memoize a measured boundary.

why: A re-render is not automatically a bug. The goal is to reduce expensive or user-visible work, not to eliminate every render.

React forms, errors, and testing

- Model forms as states: untouched, editing, submitting, success, validation error, server error, and retrying. Avoid disabling every escape route while a request is pending.

- An error boundary handles rendering errors in a subtree, not arbitrary promise rejections from event handlers. Event handlers need their own error handling.
- Test behavior through user-visible outcomes. Prefer role, label, text, and accessible name queries over implementation details.
- Test loading, success, empty, error, retry, slow response, cancellation, duplicate submission, keyboard use, and narrow layouts.

```
it('shows a recoverable error', async () => {
  render(<ProfileForm />);
  await user.click(screen.getByRole('button', {name: /save/i}));
  expect(await screen.findByRole('alert')).toHaveTextContent('Try again');
  expect(screen.getByRole('button', {name: /save/i})).toBeEnabled();
});
```

Part seven Next.js

Next.js interview answers should distinguish framework behavior from React behavior. In the App Router, the file system, server and client boundaries, caching, streaming, and deployment model affect how a page behaves.

App Router concepts

Concept	What to explain	Edge case
layout.tsx	Shared UI that persists across navigation and can nest by route segment.	A layout can retain state while a page changes; do not assume every navigation remounts it.
page.tsx	The UI for a route segment.	Dynamic params and loading or error boundaries change the data lifecycle.
Server Component	Runs on the server and can access server-only resources without shipping its implementation to the browser.	It cannot use browser event handlers or client-only hooks.
Client Component	Uses 'use client' and can use state, effects, and browser APIs.	The boundary affects the bundle and must not receive non-serializable props.
loading.tsx	Provides a route-level loading UI while a segment is pending.	A fallback must not cause content to shift or hide useful navigation.
error.tsx	Provides a route-level recovery boundary for errors.	It must be a client component and should expose a retry path without leaking details.

Concept	What to explain	Edge case
route.ts	Defines an HTTP endpoint in the App Router.	Validate input, auth, method, rate limits, and response caching on the server.

20. Why should a component remain a Server Component when possible?

answer: It can reduce client JavaScript, fetch server-side data close to the source, keep secrets on the server, and stream rendered UI without sending the component implementation to the browser.

why: Client boundaries are valuable but have a cost. Make only interactive or browser-dependent parts client components and keep the data and rendering boundary deliberate.

interviewer follow-up: Where would you place a search box inside a server-rendered page?

follow-up answer: Keep the page and data-heavy content server-rendered, then isolate the input, URL update, and interactive suggestions in a small Client Component. Let the URL be the shareable query state when appropriate.

why: A narrow client boundary preserves server benefits while giving the input the browser APIs and state it needs.

21. How do caching and revalidation affect a Next.js page?

answer: They determine whether data is reused, when it becomes stale, and whether a request runs at build time, on the server, or during a client interaction. The right policy depends on freshness, personalization, and cost.

why: A cached response can be fast but wrong for personalized or rapidly changing data. A fully dynamic response can be correct but expensive. Explain the product freshness requirement first.

interviewer follow-up: How would you handle a feed that should not insert new cards while the user is scrolling?

follow-up answer: Subscribe to a lightweight version or realtime signal, collect new ids in memory, and show a 'new echoes' control. Apply the update only after the user taps it or reaches a deliberate refresh point, then preserve scroll position.

why: Inserting content above the reading position changes what the user sees and can make them lose context. User-controlled refresh is more predictable.

- Use dynamic rendering for personalized or request-dependent data when static reuse would be unsafe or incorrect.
- Keep secrets in server-only environment variables. Public variables are intentionally shipped to the browser.

- Use next/image or an equivalent image strategy with known dimensions, responsive sizes, and safe remote-image configuration.
- Use metadata and structured URLs for SEO, but do not make SEO the reason to ship a large client bundle.
- For authentication, validate on the server and protect data access. Middleware or route guards improve routing behavior but do not replace authorization at the data boundary.
- Use loading and error boundaries as product states, not just framework defaults. Include retry, empty, offline, and permission-denied states.

```
// app/products/[id]/page.tsx
export default async function ProductPage({
  params,
}: {
  params: Promise<{id: string}>;
}) {
  const {id} = await params;
  const product = await getProduct(id);
  if (!product) notFound();
  return <ProductDetails product={product} />;
}
```

Part eight production frontend

A production frontend is a system: it must load, respond, recover, remain accessible, protect user data, and be diagnosable. These topics often separate a syntax-focused candidate from someone who can own a feature.

Performance and Core Web Vitals

Metric	What it represents	Typical improvement
LCP	How quickly the main content becomes visible.	Prioritize the hero resource, reduce server delay, optimize images and fonts, and avoid render-blocking work.
INP	How responsive the page is to user interactions.	Reduce long tasks, split JavaScript, avoid expensive synchronous work, and keep event handlers small.
CLS	How much visible content shifts unexpectedly.	Reserve image and ad space, stabilize fonts, and avoid inserting content above the current viewport.
TTFB	How quickly the first response bytes arrive.	Improve server work, caching, routing, and network geography; do not confuse it with total page load.

- Measure real users and lab runs. A fast laptop in development is not representative of a low-end phone on a slow network.
- Reserve dimensions for images, video, ads, and async UI. Loading placeholders should match the final geometry.
- Lazy-load work that is below the fold, but do not lazy-load the primary content required for the first view.
- Code-split by route and feature. A smaller initial JavaScript payload improves parse, compile, and execute time.
- Use cache invalidation, compression, modern image formats, CDN delivery, and correct cache headers without exposing private responses.

Security

Risk	Frontend defense	Server requirement
XSS	Escape output, avoid unsafe HTML, sanitize trusted rich text, use CSP and safe URL handling.	Validate and authorize; do not trust the client or a sanitized display value.
CSRF	SameSite cookies, anti-CSRF token patterns, and correct method semantics.	Verify origin or token and require authorization for mutations.
Injection	Treat user input as data and use safe APIs.	Parameterized queries and validation at the server boundary.
Supply chain	Lock dependencies, review updates, use audits and least privilege.	Build and deploy with restricted credentials and reproducible artifacts.
Token leakage	Do not ship private keys or service credentials; minimize persisted tokens.	Rotate, scope, expire, revoke, and audit credentials.

22. Why is client-side validation not enough?

answer: It improves user feedback but the client is controlled by the user and can be bypassed. The server must validate shape, permissions, business rules, and security constraints.

why: A browser is an untrusted environment. The API is the authority for data integrity and authorization.

interviewer follow-up: What would you validate in the UI, and what would you validate only on the server?

follow-up answer: The UI should validate format, required fields, obvious limits, and immediate feedback. The server must validate authorization, ownership, business rules, canonical values, rate limits, and all untrusted input again.

why: Client validation is for usability. The server is the authority because the client can be modified or bypassed.

Testing strategy

- Unit tests isolate pure functions and small state transitions. They are fast and precise but do not prove browser integration.
- Component or integration tests exercise rendering, user events, network boundaries, and accessibility states together.
- End-to-end tests verify critical flows in a real browser: login, checkout, file upload, routing, and recovery.
- Contract tests protect the shape and semantics of APIs between frontend and backend.
- Test the unhappy path. A feature is not complete if it only handles a fast successful request.

A useful test matrix: initial, loading, success, empty, validation error, server error, offline, timeout, retry, cancellation, duplicate action, permission change, refresh, back navigation, keyboard, screen reader, narrow viewport, large text, and reduced motion.

State and data architecture

Classify state before choosing a library: local UI state, server state, URL state, form state, session state, and derived state have different ownership and lifecycles. A global store is not automatically the right place for every category.

- Server state needs caching, deduplication, stale handling, retries, invalidation, pagination, and optimistic update rules.
- URL state should be shareable and navigable when it represents filters, tabs, search, or a resource identity.
- Optimistic mutations need a rollback or refetch policy. Protect against duplicate taps, out-of-order responses, and offline replay.
- For infinite lists, use stable cursors, deduplicate by identity, preserve scroll position, and avoid inserting new results while the user is reading without consent.

Frontend architecture and code review

- Organize around user-visible features or domains when that helps ownership and change locality. Avoid a folder structure that hides behavior behind generic utility layers.
- Keep rendering, data access, validation, and side effects separable enough to test. Do not create abstractions before you have a repeated problem.
- A good review checks correctness, accessibility, performance, security, error states, responsive behavior, observability, and maintenance cost.

- A design system should define tokens, component states, interaction rules, and accessibility behavior, not only colors and buttons.

Part nine practical coding questions

For coding rounds, explain the contract first, write the smallest correct version, test edge cases, then discuss complexity and production hardening. Do not rush into abstraction before the behavior is clear.

Exercise one Debounce

prompt: Implement debounce(fn, delay) so rapid calls schedule only the latest invocation. Return a cancel method. Explain what happens to this and arguments.

```
function debounce<T extends (...args: any[]) => void>(
  fn: T,
  delay: number,
) {
  let timer: ReturnType<typeof setTimeout> | undefined;

  const debounced = function (this: ThisParameterType<T>, ...args: Parameters<T>) {
    if (timer) clearTimeout(timer);
    timer = setTimeout(() => {
      timer = undefined;
      fn.apply(this, args);
    }, delay);
  };

  debounced.cancel = () => {
    if (timer) clearTimeout(timer);
    timer = undefined;
  };

  return debounced;
}
```

why this answer works: The timer is replaced for every call, so only the final call runs after a quiet period. apply preserves the caller and arguments. Cancel prevents a pending call from firing after the feature is gone.

edge cases: delay less than zero, repeated cancel calls, component cleanup, exceptions inside fn, and whether the function should support leading or trailing invocation.

Exercise two Throttle

prompt: Implement a throttle for scroll or pointer events. Explain leading and trailing behavior and why timers alone may not be enough for animation.

```
function throttle<T extends (...args: any[]) => void>(fn: T, wait: number) {
  let last = 0;
  let timer: ReturnType<typeof setTimeout> | undefined;
  let pending: Parameters<T> | undefined;

  return (...args: Parameters<T>) => {
    const now = Date.now();
    pending = args;
    const run = () => {
      last = Date.now();
      timer = undefined;
      if (pending) {
        const next = pending;
        pending = undefined;
        fn(...next);
      }
    };

    if (now - last >= wait) run();
    else if (!timer) timer = setTimeout(run, wait - (now - last));
  };
}
```

stronger production answer: For visual work, prefer requestAnimationFrame so updates align with painting. For scroll listeners, use passive listeners when you do not call preventDefault, and clean up the listener.

Exercise three Accessible modal

prompt: Build a modal that opens from a button, traps focus, closes on Escape, labels itself, restores focus, and respects reduced motion.

- Use a real dialog pattern or a tested dialog primitive rather than inventing focus behavior from scratch.
- On open, save the trigger and focus the first meaningful control. On close, restore focus if the trigger still exists.
- Prevent background interaction and announcement while open. Do not hide the dialog from assistive technology during its active state.
- Test no focus trap when the dialog has no tabbable children, nested dialogs, browser zoom, and an unmount during closing.

Exercise four Typed API client

prompt: Create a typed request function that accepts an unknown JSON response, validates it, handles HTTP failures, and exposes a discriminated result.

```
type User = {id: string; name: string};
type ApiResult<T> =
  | {ok: true; data: T}
  | {ok: false; kind: 'http' | 'network' | 'invalid'; message: string};

function isUser(value: unknown): value is User {
  if (!value || typeof value !== 'object') return false;
  const item = value as Record<string, unknown>;
  return typeof item.id === 'string' && typeof item.name === 'string';
}

async function getUser(id: string): Promise<ApiResult<User>> {
  try {
    const response = await fetch(`/api/users/${encodeURIComponent(id)}`);
    if (!response.ok) return {ok: false, kind: 'http', message: 'request failed'};
    const value: unknown = await response.json();
    return isUser(value)
      ? {ok: true, data: value}
      : {ok: false, kind: 'invalid', message: 'unexpected response'};
  } catch {
    return {ok: false, kind: 'network', message: 'network unavailable'};
  }
}
```

why this answer works: The external response begins as unknown. The guard establishes the runtime fact before the rest of the application relies on the type. The result separates expected failure categories without leaking raw technical details to users.

Exercise five React autocomplete

prompt: Build an autocomplete with debounce, cancellation, keyboard navigation, loading, empty, error, and selection states.

- Keep the input value, highlighted index, request state, and selected item distinct. Do not use the displayed label as identity if ids exist.
- Use role combobox, aria-expanded, aria-controls, option roles, aria-activedescendant, and a stable listbox id.
- Do not announce every keystroke as an error. Announce result counts or failures in a polite live region when useful.
- Abort obsolete requests and protect against a response arriving after the user selected an item or closed the list.

Exercise six Next.js feed

prompt: Design a feed in Next.js where the first page is fast, the next page uses a stable cursor, new items are collected in a banner while the user scrolls, and mutations work offline.

- Render the initial page on the server when that helps first content, but keep interactive pagination and the new-items banner in a focused client boundary.

- Use a cursor based on a stable ordering such as score, createdAt, and id. Avoid offset pagination when records can arrive or change between requests.
- Keep a per-session exploration seed if ranking includes exploration, so page boundaries do not reshuffle every request.
- Store pending mutations in an outbox with an idempotency key, retry policy, conflict handling, and a visible offline state.
- Deduplicate by record id and never silently pretend a failed mutation succeeded after the server rejects it.

Part ten multiple choice questions

Choose an answer before reading the explanation. The answer is intentionally followed by why and, where useful, the edge case that changes the decision.

1. Which operation is usually the most appropriate for reading a cacheable resource?

- A. POST
- B. GET
- C. PATCH
- D. DELETE

correct answer: B. GET

why: GET is safe by convention and can be cached when response headers allow it.

2. What does event delegation rely on?

- A. Capture only
- B. Bubbling and a stable ancestor
- C. CSS selectors only
- D. A Web Worker

correct answer: B. Bubbling and a stable ancestor

why: The ancestor receives bubbled events and can inspect the target. It reduces listener count for dynamic lists.

3. Which CSS property is generally the safest starting point for smooth movement?

- A. top
- B. margin-left
- C. transform
- D. width

correct answer: C. transform

why: Transforms can often be composited without layout, although the whole scene still needs profiling.

4. What is the main problem with index keys in a reorderable list?

- A. They are invalid syntax
- B. They force a network request
- C. Identity can move between items
- D. They disable CSS

correct answer: C. Identity can move between items

why: React uses keys to match item identity across renders. An index changes meaning when order changes.

5. What does fetch do for a 404 response?

- A. Always rejects
- B. Resolves with a non-ok response
- C. Retries forever
- D. Converts it to null

correct answer: B. Resolves with a non-ok response

why: Network failure and HTTP failure are different. Application code should check response.ok or status.

6. Which type is safest for unvalidated JSON?

- A. any
- B. unknown
- C. never
- D. void

correct answer: B. unknown

why: unknown forces narrowing before use. It preserves the uncertainty at the runtime boundary.

7. What does strictNullChecks change?

- A. It removes null from JavaScript
- B. It makes null and undefined explicit in types
- C. It validates API JSON
- D. It prevents all runtime errors

correct answer: B. It makes null and undefined explicit in types

why: It catches missing checks at compile time, but it cannot validate data received at runtime.

8. What is a good reason to use useReducer?

- A. Every component must use it
- B. Related state transitions are easier to model as actions
- C. It makes network calls synchronous
- D. It avoids all re-renders

correct answer: B. Related state transitions are easier to model as actions

why: Reducers make explicit transitions useful when state has several related fields or event types.

9. Which statement about useEffect is strongest?

- A. Use it for every derived value
- B. Use it to synchronize with external systems
- C. It runs before render
- D. It replaces event handlers

correct answer: B. Use it to synchronize with external systems

why: Effects are for timers, subscriptions, browser APIs, and imperative systems, not ordinary pure calculations.

10. What is the best first response to a Core Web Vital regression?

- A. Add more memo calls
- B. Measure the affected flow and identify the bottleneck
- C. Disable analytics
- D. Increase every timeout

correct answer: B. Measure the affected flow and identify the bottleneck

why: Performance work without measurement often optimizes the wrong layer.

11. Which is a server responsibility even when the UI hides a button?

- A. Font loading
- B. Authorization
- C. CSS specificity
- D. Focus ring color

correct answer: B. Authorization

why: The client is untrusted. The server must check the current user and permission for every protected operation.

12. What does a SameSite cookie attribute help with?

- A. Image compression

- B. Cross-site cookie sending behavior
- C. Type inference
- D. React reconciliation

correct answer: B. Cross-site cookie sending behavior

why: SameSite helps reduce some CSRF patterns, but cookie and mutation design still need care.

13. Which Next.js component can use browser event handlers?

- A. Any Server Component
- B. A Client Component
- C. layout.tsx only
- D. route.ts only

correct answer: B. A Client Component

why: State, effects, and browser APIs require a client boundary.

14. Why reserve image dimensions?

- A. To improve TypeScript
- B. To avoid layout shift
- C. To enable CORS
- D. To reduce HTTP status codes

correct answer: B. To avoid layout shift

why: The browser can allocate geometry before the image arrives, reducing unexpected movement.

15. Which test query is usually strongest for an interactive button?

- A. A generated class name
- B. A DOM implementation detail
- C. Its accessible role and name
- D. The component function name

correct answer: C. Its accessible role and name

why: It tests how the user and assistive technology identify the control and survives many refactors.

16. What does Promise.allSettled provide?

- A. Only the first value
- B. Results for every promise regardless of fulfillment
- C. Automatic retries
- D. A transaction rollback

correct answer: B. Results for every promise regardless of fulfillment

why: It is useful when independent tasks should report separately.

17. What is a stale closure?

- A. A closed browser tab
- B. A callback that retains an older value from its creation scope
- C. A CSS selector
- D. A cached image

correct answer: B. A callback that retains an older value from its creation scope

why: Dependency choices, functional updates, and refs can address different stale-value cases.

18. What does a CSP primarily help restrict?

- A. Which scripts and resources may run or load
- B. Which CSS grid tracks exist
- C. Which TypeScript types compile
- D. Which HTTP method is idempotent

correct answer: A. Which scripts and resources may run or load

why: CSP is a defense-in-depth control against some injection and resource-loading attacks.

19. Which pagination choice is more stable when records arrive between requests?

- A. OFFSET only
- B. A cursor based on stable ordering
- C. Random page numbers
- D. Client array indexes

correct answer: B. A cursor based on stable ordering

why: A cursor can continue after the last known record instead of relying on shifting row positions.

20. What does a TypeScript type assertion do at runtime?

- A. Validates the value
- B. Converts the value
- C. Changes only the compiler's view
- D. Serializes the value

correct answer: C. Changes only the compiler's view

why: Runtime validation requires actual checks or a schema parser.

Part eleven real world scenario questions

Scenario 1 The page is fast on your laptop but slow on low-end Android phones.

strong response: Measure on a throttled device, inspect JavaScript parse and execution, long tasks, image size, layout work, and network timing. Reduce initial work, split code, reserve image geometry, defer non-critical work, and verify real-user data. Do not claim that React memoization alone solves it.

what this tests: Ask which user journey and metric regressed. A good answer connects the fix to measurement.

Scenario 2 An autocomplete sometimes shows results for the previous query.

strong response: Debounce input to reduce requests, abort obsolete requests, and commit only the response associated with the current query or request id. Show loading and error states without clearing useful text unnecessarily.

what this tests: The race is caused by response order, not merely by typing speed.

Scenario 3 A dashboard has ten widgets, and one API fails.

strong response: Model each widget as independent if the product allows partial success. Use allSettled-like behavior or separate queries, show local retry, and preserve successful data. If the dashboard is a transaction, define the all-or-nothing contract instead.

what this tests: The correct choice follows the user task, not a universal rule.

Scenario 4 A feed duplicates items on page two and sometimes skips items.

strong response: Replace offset pagination with a stable cursor based on the ranking tuple and a session-stable exploration seed. Deduplicate by id as a safety net, not as the ordering algorithm. Capture new items in a banner instead of inserting while the user reads.

what this tests: Mention concurrent inserts, score changes, deletes, and retries.

Scenario 5 A user reports that a dialog cannot be dismissed with a keyboard.

strong response: Reproduce with Tab, check focus placement, focus trap, Escape handling, native semantics, and focus restoration. Add an automated keyboard and accessibility test, then verify on a real screen reader when possible.

what this tests: Accessibility bugs are interaction bugs, not cosmetic bugs.

Scenario 6 A product manager asks to store an access token in localStorage.

strong response: Explain the XSS exposure and the threat model. Prefer an appropriately scoped secure session strategy, short-lived tokens, rotation, HttpOnly cookies where compatible, and server-side revocation. If localStorage is accepted for a constrained reason, document the risk and harden the application rather than calling it secure.

what this tests: Avoid absolute claims; security is a design and threat-model decision.

Scenario 7 A Next.js page shows different content after hydration.

strong response: Find non-deterministic render inputs such as Date, Math.random, locale, browser-only APIs, or data that changed between server and client. Move browser-dependent work behind a client boundary or render a stable initial value and update after mount.

what this tests: Do not hide the warning without fixing the mismatch.

Scenario 8 An optimistic like button feels instant but becomes wrong after a failed request.

strong response: Update immediately with a request id or mutation key, disable duplicate mutation only when appropriate, show a pending state, roll back or refetch on failure, and reconcile with the server response. Handle offline queueing separately from an online error.

what this tests: The user should never be left with a false confirmed state.

Scenario 9 A CSS dropdown is hidden behind a card even with z-index: 9999.

strong response: Inspect stacking contexts created by transforms, opacity, filters, positioned ancestors, isolation, and containment. Move the overlay to a suitable portal or change the context deliberately. Then test clipping, focus, scroll, and mobile viewport behavior.

what this tests: The numeric z-index is not global across stacking contexts.

Scenario 10 A release has a large bundle after adding one chart library.

strong response: Use bundle analysis to find the imported modules, check tree-shaking, dynamic import the chart, use a lighter build or server rendering where appropriate, and remove unused locales or adapters. Measure the initial route, not only total bundle size.

what this tests: A smaller total bundle is not enough if the initial route still loads the heavy module.

Part twelve debugging drills

23. A component fetches twice in development. Is that automatically a production bug?

answer: Not automatically. React development checks can expose missing cleanup or impure effects. Investigate whether the effect is safe to repeat, whether requests are aborted, and whether production behavior and user-visible side effects are correct.

why: The right fix is idempotent setup and cleanup, not disabling a development check. A duplicate payment or mutation would still be a serious design problem.

interviewer follow-up: How would you distinguish a development-only diagnostic from a real duplicate mutation?

follow-up answer: Compare development and production behavior, inspect the request log, check whether the effect has cleanup, and add an idempotency key or server-side deduplication for the mutation. Reproduce with a real user action rather than assuming a framework check is the cause.

why: Development diagnostics can expose unsafe effects, but payments, posts, and deletes must be safe even if a client retries or a process is interrupted.

24. A CSS animation jitters when the browser is resized. What do you inspect?

answer: Check layout dependencies, changing element dimensions, font loading, viewport units, scrollbars, transitions on layout properties, keyframe interpolation, and whether React remounts the animated element.

why: Jitter is usually a changing input or a competing animation owner. Keep one source of truth for the animation and reserve geometry.

interviewer follow-up: How would you make the animation accessible for reduced motion?

follow-up answer: Wrap non-essential transitions in a prefers-reduced-motion media query and replace large movement with an instant state change or a subtle opacity change. Never hide required status information when motion is reduced.

why: Reduced motion is a user preference that can prevent discomfort. The information and interaction must remain available without animation.

25. A form loses input when a validation request fails. What is wrong?

answer: The UI likely replaced the form state with server data or remounted the form on every state transition. Keep draft input separate from response state, preserve values on recoverable errors, and use stable keys.

why: Form drafts are user-owned state. An error response should update status and field messages, not erase work unless the product explicitly says so.

interviewer follow-up: How would you handle a server response that normalizes the saved value?

follow-up answer: Replace the optimistic draft with the canonical server value, preserve focus and the user's current workflow, and show any normalization clearly when it changes what they typed. Keep the mutation id so an older response cannot overwrite a newer edit.

why: The server may trim, format, assign ids, or apply policy. The client must reconcile with that authority without causing a visible jump or losing newer local work.

26. A page has good Lighthouse scores but users complain it is slow. What next?

answer: Compare lab and real-user data, segment by device, geography, network, route, and interaction. Look for post-load work, slow input handling, third-party scripts, backend latency, and errors not represented in the lab run.

why: Lab tests are controlled samples. Real-user performance includes device diversity and the full product session.

interviewer follow-up: Which user-facing metric would you set as a release guardrail?

follow-up answer: Choose a metric tied to the critical task, such as p75 INP for interaction responsiveness, p75 LCP for first content, CLS for layout stability, error-free task completion, or conversion. Define the segment and threshold before release.

why: A single global score can hide a regression on low-end devices or a key route. A task metric creates an actionable release decision.

Part thirteen interview communication

Technical correctness is necessary, but interviewers also evaluate how you think. A clear answer is easier to trust and easier to build with.

- Clarify the contract before coding: inputs, outputs, constraints, empty behavior, failure behavior, and expected scale.
- State a simple approach first. Then improve it if the constraints require better time, space, accessibility, or operational behavior.
- Narrate decisions, not every keystroke. Say what invariant you are preserving and what test case you are checking.
- When you do not know, say what you would verify and how you would reduce uncertainty. Guessing confidently is less useful than a sound investigation plan.
- For behavioral questions, use context, responsibility, action, result, and reflection. Include a concrete tradeoff or lesson.

Questions to ask the interviewer

- What does the team own end to end, and how do frontend engineers participate in product and API decisions?
- How do you measure frontend quality: performance, accessibility, reliability, conversion, or support volume?
- What are the largest frontend constraints today: legacy code, scale, design system, release speed, or browser support?
- How are incidents, regressions, and technical debt handled after a feature ships?
- What would a successful first three months look like in this role?

Part fourteen final revision sheet

Area	Be ready to explain	Practice task
HTML	Semantics, forms, labels, focus, SEO, media, and accessibility.	Build a keyboard-friendly form without a framework.
CSS	Cascade, box model, flex, grid, responsive constraints, stacking, and animation.	Recreate a responsive card grid with no fixed page height.
JavaScript	Closures, this, prototypes, equality, promises, event loop, modules, and errors.	Predict event-loop output and implement debounce plus cancellation.
TypeScript	Unknown, narrowing, generics, unions, utility types, strict mode, and runtime validation.	Type an API client that validates unknown JSON.
React	Render model, keys, state ownership, effects, hooks, forms, performance, and testing.	Build an autocomplete with loading, error, keyboard, and race protection.
Next.js	Server/client boundaries, routing, data, caching, loading, errors, auth, and deployment.	Design a cursor-paginated feed with a new-items banner.
Production	Performance, security, accessibility, testing, observability, and recovery.	Describe the release plan and failure states for a payment or upload flow.

Last hour before the interview: Review the event loop, CSS layout, React effects and keys, TypeScript runtime boundaries, Next.js server and client components, and one project story with measurable impact. Then stop studying and arrive rested.

Part fifteen compact code examples and why

These small examples connect the interview concepts to code. Read the code, then explain the invariant it preserves and the edge case that would change the design.

Event delegation

```
document.querySelector('#list')?.addEventListener('click', (event) => {
  const target = event.target;
  if (!(target instanceof HTMLElement)) return;
  const button = target.closest('[data-action=delete]');
  if (!button) return;
  deleteItem(button.dataset.id);
});
```

Why: one listener handles current and future rows, while closest keeps the behavior attached to the semantic control.

Safe fetch result handling

```
async function readJson<T>(response: Response): Promise<T> {
  if (!response.ok) {
    throw new Error(`http ${response.status}`);
  }
  return (await response.json()) as T;
}

const data = await readJson<User[]>(await fetch('/api/users'));
```

Why: fetch does not reject for HTTP errors. Checking ok keeps transport failure and application failure from being confused.

Accessible HTML form

```
<label for="email">Email address</label>
<input id="email" name="email" type="email"
  autocomplete="email" aria-describedby="email-error" />
<p id="email-error" role="alert"></p>
```

Why: the label gives the control a name, autocomplete helps input, and the described error can be announced without replacing the field.

Responsive CSS grid

```
.results {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 18rem), 1fr));
  gap: 1rem;
  padding-inline: max(1rem, 4vw);
}
```

Why: the grid adapts to its available width, wraps long content, and keeps safe side spacing without device-specific breakpoints.

Yielding long JavaScript work

```
async function processInBatches(items: Item[]) {
  for (let i = 0; i < items.length; i += 100) {
    process(items.slice(i, i + 100));
    await new Promise(requestAnimationFrame);
  }
}
```

Why: yielding gives the browser a chance to paint and accept input. For CPU-heavy work, a Web Worker may be a better boundary.

Bounded concurrency

```
async function mapWithLimit<T, R>(items: T[], limit: number, fn: (item: T) =>
Promise<R>) {
  const results: R[] = [];
  let next = 0;
  async function worker() {
    while (next < items.length) {
      const index = next++;
      results[index] = await fn(items[index]);
    }
  }
  await Promise.all(Array.from({length: Math.min(limit, items.length)}, worker));
  return results;
}
```

Why: the worker count bounds requests, memory, and decoding pressure while preserving result order.

Runtime validation at a TypeScript boundary

```
function isPage(value: unknown): value is {items: User[]; next: string | null} {
  if (!value || typeof value !== 'object') return false;
  const page = value as Record<string, unknown>;
  return Array.isArray(page.items) &&
    (typeof page.next === 'string' || page.next === null);
}
```

Why: TypeScript types disappear at runtime. A guard makes the external response safe before application code uses it.

React functional update

```
setCount(current => current + 1);
```

Why: the updater reads the latest queued state, so multiple events do not overwrite each other through a stale closure.

React controlled input

```
const [value, setValue] = useState('');
return (
  <input value={value} onChange={event => setValue(event.target.value)} />
);
```

Why: React owns the current value, which makes validation and dependent UI predictable. For very large forms, uncontrolled fields can reduce render work.

Next.js server and client boundary

```
// app/search/page.tsx
export default async function Page({searchParams}) {
  const results = await search(searchParams.q ?? '');
  return <SearchResults initialResults={results} />;
}

// SearchResults.tsx
'use client';
export function SearchResults({initialResults}) {
  const [query, setQuery] = useState('');
  return <SearchBox query={query} onChange={setQuery} results={initialResults} />;
}
```

Why: data fetching stays on the server while the small interactive boundary owns browser state and event handlers.

Stable cursor pagination

```
const cursor = encodeURIComponent(`${score}|${createdAt}|${id}`);
const response = await fetch(`/api/feed?cursor=${cursor}`);
```

Why: a cursor continues from a stable ordering tuple, so inserts and score changes do not shift page positions as badly as OFFSET pagination.

Testing user-visible behavior

```
await user.click(screen.getByRole('button', {name: /save/i}));
expect(await screen.findByRole('status')).toHaveTextContent('Saved');
```

Why: role and name queries test the contract a user and assistive technology can observe instead of a component's private implementation.

Performance measurement

```
performance.mark('search-start');
await loadSearchResults();
performance.mark('search-end');
performance.measure('search', 'search-start', 'search-end');
```

Why: marks give a concrete duration that can be compared before and after a change instead of relying on intuition.

Safe text rendering

```
return <p>{userProvidedText}</p>;

// Avoid rendering untrusted HTML unless it is sanitized by a trusted policy.
// dangerouslySetInnerHTML is an explicit security boundary.
```

Why: normal JSX text is escaped by React. Raw HTML requires a strict sanitization and review process because it can create XSS.

Reference material

These primary references were checked while preparing this handbook. Documentation changes over time, so use the relevant official page when a framework version or browser behavior matters.

MDN web developer guides - https://developer.mozilla.org/en-US/docs/MDN/Guides. HTML, CSS, JavaScript, APIs, performance, and browser platform guidance.

MDN JavaScript guide - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide. Language fundamentals, promises, modules, iterators, memory, and advanced behavior.

React reference - https://react.dev/reference/react. Hooks, components, APIs, and React-specific behavior.

TypeScript Handbook - https://www.typescriptlang.org/docs/handbook/. Everyday types, narrowing, functions, objects, generics, modules, and utility types.

Next.js App Router - https://nextjs.org/docs/app. Routing, layouts, Server Components, Suspense, Server Functions, and navigation.

Next.js installation and project structure - https://nextjs.org/docs/app/getting-started/installation. Current App Router setup, conventions, supported browsers, TypeScript, and tooling.

web.dev Learn Performance - https://web.dev/learn/performance. Performance measurement, loading, rendering, and Core Web Vitals concepts.

web.dev Learn Accessibility - https://web.dev/learn/accessibility. Accessible structure, interaction, and testing guidance.

OWASP Cross Site Scripting Prevention - https://owasp.org/www-community/attacks/xss/. Threat model and defense concepts for cross-site scripting.

Playwright documentation - https://playwright.dev/docs/intro. Browser automation and end-to-end testing concepts.

Testing Library guiding principles - https://testing-library.com/docs/guiding-principles/. Testing user-visible behavior instead of implementation details.

Use this handbook as a reasoning guide, not a script. The strongest candidate can explain a simple answer, name its limits, test the edge cases, and choose a different approach when the product constraints change.